OSSP Skill Week -8

2520090058:

Task 1-: To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
  UW PICO 5.09                                File: fff.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int fd;
    pid_t pid;

    printf("=== Combined Redirection Demo ===\n");

    /*
     * Open output file.
     * O_TRUNC clears old contents.
     */
    fd = open("combined_output.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        /*
         * Duplicate fd to stdout.
         * Equivalent to:
         * stdout > combined_output.txt
         */
        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2 stdout");
            close(fd);
            exit(1);
        }

        /*
         * Duplicate fd to stderr.
         * Equivalent to:
         * stderr 2>&1
         *
         * Both stdout and stderr now point
         * to the same file.
         */
        if (dup2(fd, STDERR_FILENO) == -1) {
            perror("dup2 stderr");
            close(fd);
            exit(1);
        }

        close(fd);

        /*
         * Generate output on both streams.
         * Because stdout is buffered, flush it
         * before writing to stderr.
         */
        printf("Child: Standard output message 1\n");
        fflush(stdout);

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg  ^U UnCut Text ^T To Spell
```

Output:

```
Last login: Wed Sep 30 08:43:55 on ttys000
sreenikhilreddy@Srees-MacBook-Pro ~ % nano fff.c

sreenikhilreddy@Srees-MacBook-Pro ~ % gcc fff.c -o fff
sreenikhilreddy@Srees-MacBook-Pro ~ % ./fff
=== Combined Redirection Demo ===

Child process completed.

=== Contents of combined_output.txt ===
Child: Standard output message 1
Child: Standard error message 1
Child: Standard output message 2
Child: Standard error message 2

=== Test Completed Successfully ===
sreenikhilreddy@Srees-MacBook-Pro ~ %
```

Task-2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
 UW PICO 5.09                                File: aaa.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_LENGTH 1024

/* Remove leading and trailing spaces */
void trim(char *str) {
    char *start = str;

    while (*start == ' ' || *start == '\t')
        start++;

    memmove(str, start, strlen(start) + 1);

    int len = strlen(str);

    while (len > 0 &&
           (str[len - 1] == ' ' || str[len - 1] == '\t')) {
        str[len - 1] = '\0';
        len--;
    }
}

/* Parse command into arguments */
int parse_arguments(char *command, char *args[]) {
    int count = 0;

    char *token = strtok(command, " \t");

    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;

    return count;
}

/* Check basic syntax */
int validate_syntax(char *input) {

    if (strlen(input) == 0) {
        return 0;
    }

    /* Command cannot start or end with pipe */
    if (input[0] == '|' ||
        input[strlen(input) - 1] == '|') {
        printf("Syntax Error: Invalid pipe position.\n");
        return 0;
    }

    /* Check consecutive pipes */
    if (strstr(input, "||") != NULL) {
        printf("Syntax Error: Consecutive pipes found.\n");
        return 0;
    }

    /* Check redirection operators */
    if (strstr(input, ">>") != NULL ||
        strstr(input, ">") != NULL ||
        strstr(input, "<") != NULL ||

^G Get Help      ^O WriteOut      ^R Read File     ^Y Prev Pg       ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where is      ^V Next Pg       ^U UnCut Text    ^T To Spell
```

Output:

```
sreenikhilreddy@Srees-MacBook-Pro ~ % nano aaa.c
sreenikhilreddy@Srees-MacBook-Pro ~ % gcc aaa.c -o aaa
sreenikhilreddy@Srees-MacBook-Pro ~ % ./aaa
========================================
       COMPLEX COMMAND MINI SHELL
========================================

Supported features:
  |   Pipe
  <   Input redirection
  >   Output redirection
  >>  Append redirection
  2>  Error redirection

Examples:
  cat input.txt | grep hello
  cat input.txt | grep hello > output.txt
  sort < input.txt | uniq > result.txt
  ls -l | grep .c > files.txt

Enter command: exit

========== EXECUTION PLAN ==========
Command 1: exit
====================================
execvp: No such file or directory

Command execution completed.
sreenikhilreddy@Srees-MacBook-Pro ~ %
```

Task 3 -: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

```
 UW PICO 5.09                                   File: sss.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20
#define MAX_CMD 256

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_CMD];
    int active;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

/* Remove newline */
void remove_newline(char *str) {
    str[strcspn(str, "\n")] = '\0';
}

/* Remove leading/trailing spaces */
void trim(char *str) {
    char *start = str;

    while (*start == ' ' || *start == '\t')
        start++;

    memmove(str, start, strlen(start) + 1);

    int len = strlen(str);

    while (len > 0 &&
           (str[len - 1] == ' ' || str[len - 1] == '\t')) {
        str[len - 1] = '\0';
        len--;
    }
}

/* Check whether command is a background command */
int is_background(char *command) {
    int len = strlen(command);

    while (len > 0 &&
           (command[len - 1] == ' ' ||
            command[len - 1] == '\t')) {
        len--;
    }

    if (len > 0 && command[len - 1] == '&') {
        command[len - 1] = '\0';
        trim(command);
        return 1;
    }

    return 0;
}

/* Store background job */
void add_job(pid_t pid, char *command) {

    if (job_count >= MAX_JOBS) {
        printf("Job limit reached.\n");
        return;

^G Get Help     ^O WriteOut     ^R Read File    ^Y Prev Pg      ^K Cut Text     ^C Cur Pos
^X Exit         ^J Justify      ^W Where is     ^V Next Pg      ^U UnCut Text   ^T To Spell
```

Output:

```
sreenikhilreddy@Srees-MacBook-Pro ~ % nano sss.c
sreenikhilreddy@Srees-MacBook-Pro ~ % gcc sss.c -o sss
sreenikhilreddy@Srees-MacBook-Pro ~ % ./sss
========================================
       BACKGROUND JOB MANAGEMENT SHELL
========================================
Use '&' to run a command in background.
Use 'jobs' to display background jobs.
Use 'exit' to quit.

myshell>
```

Task 4-: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
 UW PICO 5.09                                                File: ..c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>

#define MAX_JOBS 20
#define MAX_CMD 256

/* Job states */
typedef enum {
    JOB_RUNNING,
    JOB_STOPPED,
    JOB_DONE,
    JOB_TERMINATED
} JobState;

/* Job record */
typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[MAX_CMD];
    JobState state;
} Job;

/* Global job table */
Job job_table[MAX_JOBS];
int job_count = 0;


/* Convert state to string */
const char *state_to_string(JobState state) {

    switch (state) {

        case JOB_RUNNING:
            return "Running";

        case JOB_STOPPED:
            return "Stopped";

        case JOB_DONE:
            return "Done";

        case JOB_TERMINATED:
            return "Terminated";

        default:
            return "Unknown";
    }
}


/* Add a job to the job table */
int add_job(pid_t pid, pid_t pgid, char *command) {

    if (job_count >= MAX_JOBS) {
        printf("Error: Job table is full.\n");
        return -1;
    }

    job_table[job_count].job_id = job_count + 1;

    job_table[job_count].pid = pid;

^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Pg    ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is     ^V Next Pg    ^U UnCut Text  ^T To Spell
```

OUTPUT:

```
=====================================================
                    JOB TABLE
=====================================================
ID    PID         PGID        STATE           COMMAND
-----------------------------------------------------
1     1579        1579        Running         sleep-job-1
2     1580        1580        Running         sleep-job-2
3     1581        1581        Running         sleep-job-3
=====================================================
Child finished: PID=1579

[1] Process completed: sleep-job-1

--- Job Table at 5 second(s) ---

=====================================================
                    JOB TABLE
=====================================================
ID    PID         PGID        STATE           COMMAND
-----------------------------------------------------
1     1579        1579        Done            sleep-job-1
2     1580        1580        Running         sleep-job-2
3     1581        1581        Running         sleep-job-3
=====================================================
Removing job [1] from job table.

--- Job Table at 6 second(s) ---

=====================================================
                    JOB TABLE
=====================================================
ID    PID         PGID        STATE           COMMAND
-----------------------------------------------------
1     1580        1580        Running         sleep-job-2
2     1581        1581        Running         sleep-job-3
=====================================================

--- Job Table at 7 second(s) ---

=====================================================
                    JOB TABLE
=====================================================
ID    PID         PGID        STATE           COMMAND
-----------------------------------------------------
1     1580        1580        Running         sleep-job-2
2     1581        1581        Running         sleep-job-3
=====================================================
Child finished: PID=1580

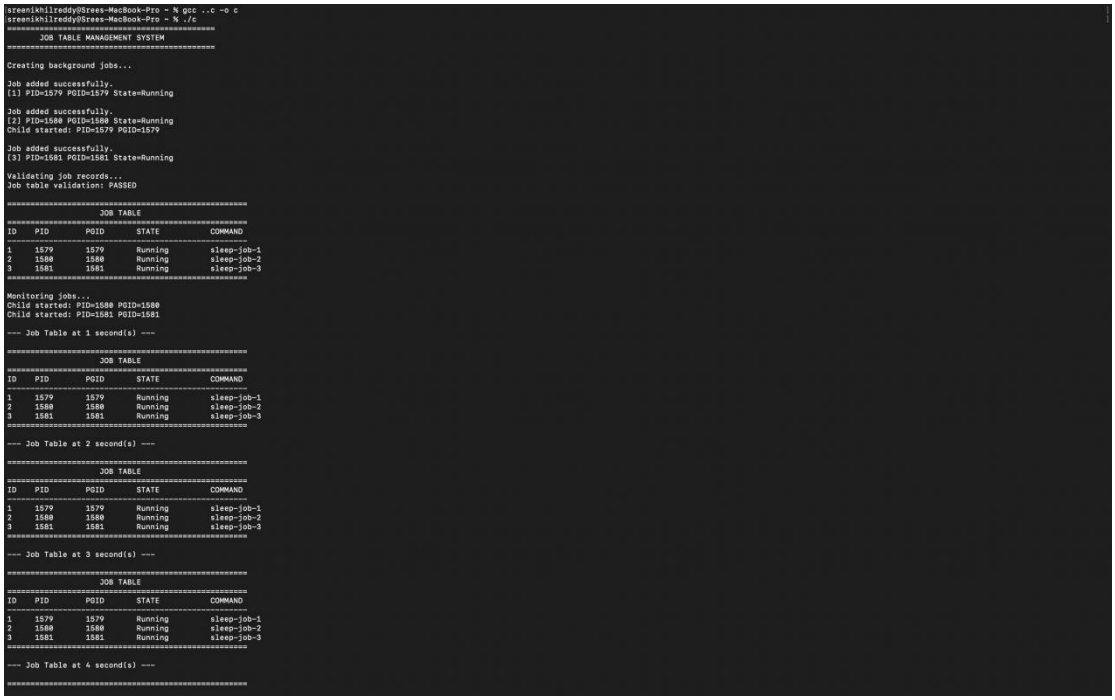
[1] Process completed: sleep-job-2

--- Job Table at 8 second(s) ---

=====================================================
                    JOB TABLE
=====================================================
ID    PID         PGID        STATE           COMMAND
-----------------------------------------------------
1     1580        1580        Done            sleep-job-2
2     1581        1581        Running         sleep-job-3
=====================================================
Removing job [1] from job table.

--- Job Table at 9 second(s) ---

=====================================================
                    JOB TABLE
=====================================================
ID    PID         PGID        STATE           COMMAND
-----------------------------------------------------
1     1581        1581        Running         sleep-job-3
=====================================================
```



```
sreenikhilreddy@Srees-MacBook-Pro ~ % gcc ..c -o c
sreenikhilreddy@Srees-MacBook-Pro ~ % ./c
==========================================
      JOB TABLE MANAGEMENT SYSTEM
==========================================

Creating background jobs...

Job added successfully.
[1] PID=1579 PGID=1579 State=Running

Job added successfully.
[2] PID=1580 PGID=1580 State=Running
Child started: PID=1579 PGID=1579

Job added successfully.
[3] PID=1581 PGID=1581 State=Running

Validating job records...
Job table validation: PASSED

==================================================
                    JOB TABLE
==================================================
ID    PID       PGID      STATE          COMMAND
--------------------------------------------------
1     1579      1579      Running        sleep-job-1
2     1580      1580      Running        sleep-job-2
3     1581      1581      Running        sleep-job-3
==================================================

Monitoring jobs...
Child started: PID=1580 PGID=1580
Child started: PID=1581 PGID=1581

--- Job Table at 1 second(s) ---

==================================================
                    JOB TABLE
==================================================
ID    PID       PGID      STATE          COMMAND
--------------------------------------------------
1     1579      1579      Running        sleep-job-1
2     1580      1580      Running        sleep-job-2
3     1581      1581      Running        sleep-job-3
==================================================

--- Job Table at 2 second(s) ---

==================================================
                    JOB TABLE
==================================================
ID    PID       PGID      STATE          COMMAND
--------------------------------------------------
1     1579      1579      Running        sleep-job-1
2     1580      1580      Running        sleep-job-2
3     1581      1581      Running        sleep-job-3
==================================================

--- Job Table at 3 second(s) ---

==================================================
                    JOB TABLE
==================================================
ID    PID       PGID      STATE          COMMAND
--------------------------------------------------
1     1579      1579      Running        sleep-job-1
2     1580      1580      Running        sleep-job-2
3     1581      1581      Running        sleep-job-3
==================================================

--- Job Table at 4 second(s) ---

==================================================
```